Astra Notes Refined Requirement Baseline

Student: Jonathan Fong
Date: April 4 2026
Project: AstraNotes

Part 1 — Refined Requirement Baseline

Every row carries the original requirement identifier where one existed, or a new identifier for requirements added during this review. The Type column explicitly classifies each as Functional, Non-Functional, or Security. The final column summarizes what changed and why.

FR = Functional Requirement | NFR = Non-Functional Requirement | SPR = Security/Privacy Requirement

ID	Type	Requirement Statement (Refined)	What Changed / Why
FR-1	Functional	The system shall create a TextNote, VoiceNote, or SecureNote identified by a system-generated UUID (RFC 4122 v4) and a user-supplied title of 1–255 non-whitespace characters. Creation is performed exclusively through NoteFactory.	Added RFC 4122 v4 constraint on UUID format; added max title length (255 chars); made NoteFactory the sole creation path explicit.
FR-1a	Functional	On title validation failure (empty, whitespace-only, or exceeding 255 characters), the system shall return a descriptive error code and leave the note collection unchanged.	Extracted title-validation rule from FR-1 into its own requirement; added 255-char upper bound as an explicit edge case.
FR-2	Functional	The system shall allow updating the title or body of an existing TextNote. Both fields obey the same title-length rule (FR-1a). On success the last-modified timestamp shall be updated; on any failure the note shall remain fully unchanged (no partial update).	Explicitly requires atomic update semantics — if update fails, note is unchanged. Added whitespace-only check to match FR-1a consistently.
FR-3	Functional	The system shall delete a note by UUID. Deletion shall remove the note from the in-memory collection and release all associated resources. For SecureNote, any in-memory key material and decryption buffers shall be zeroed before deallocation. A missing UUID shall return a failure status without throwing.	Added zeroing of key material for SecureNote on deletion; made resource-release obligation explicit for RAII compliance.
FR-4	Functional	The system shall persist all notes to a single local JSON file at a configurable path. On shutdown the entire collection shall be serialized. On startup the file shall be deserialized and each note reconstructed via NoteFactory into its correct concrete type. If two notes with the same UUID are found in the file, the second occurrence shall be discarded and the conflict logged.	Added UUID-collision-on-load handling (previously undefined). Made file path configurable rather than hardcoded.
FR-4a	Functional	If the storage file is absent at startup, the system shall start with an empty collection and create a new file on the next save. A missing file is not an error.	Previously undefined behavior — missing file was neither documented as error nor as normal. Now explicit.

ID	Type	Requirement Statement (Refined)	What Changed / Why
FR-4b	Functional	If the storage file is present but unreadable or structurally corrupt, the system shall rename it to <filename>.quarantine.<timestamp>, log the event, and start with an empty collection. Individual malformed note records within an otherwise valid file shall be skipped individually; all valid records shall still load.	Strengthened quarantine: (a) timestamp in quarantine filename avoids overwrite; (b) per-record skip added for partially-corrupt files — original only handled whole-file corruption.
FR-5	Functional	The system shall allow reading the decrypted content of a SecureNote only when supplied with the correct passphrase. The passphrase shall be at least 8 characters. Plaintext shall not be retained in any data member, log entry, or temporary buffer after the access operation completes; the decryption buffer shall be zeroed immediately after use.	Added minimum passphrase length (8 chars). Added explicit requirement to zero decryption buffer — previously only 'no persistent data member' was stated.
FR-5a	Functional	On passphrase failure (wrong, empty, or below minimum length), the system shall return a typed error code distinguishing wrong-passphrase from invalid-input. No partial plaintext shall be accessible.	Previously all failures returned a single failure status — callers could not distinguish user error from wrong passphrase.
FR-6	Functional	The system shall search the in-memory note collection by title substring and return all matching notes. Search shall be case-insensitive. No file I/O shall occur during search. An empty query shall return the full collection, not an error.	Case-sensitivity resolved: case-insensitive (deferred in original). Added empty-query behavior.
NFR-1	Non-Functional	UUID-keyed note lookup shall complete in O(1) average time using an unordered_map or equivalent hash structure. The data structure shall be the sole authoritative in-memory store.	Added 'sole authoritative store' — previously two stores could coexist silently.
NFR-2	Non-Functional	The system shall not crash or invoke undefined behavior (as detected by AddressSanitizer in debug builds) under any single-operation input, including empty strings, maximum-length strings, and missing UUIDs.	Changed from 'no crash' to 'no undefined behavior under ASan' — testable and precise.
NFR-3	Non-Functional	All owning pointers shall use unique_ptr. shared_ptr is permitted only where shared ownership is documented with an explicit rationale in the relevant header. Raw owning pointers are prohibited.	Added shared_ptr exception with mandatory rationale — original said 'unique_ptr throughout' but the backlog mentioned possible exceptions without a gate.
NFR-4	Non-Functional	Any third-party library adopted shall be documented in planning/library-decision.md with: C++17 compatibility evidence, GTest linkage test result, license classification, and supply-chain risk assessment. No library may be used without this file present and committed.	Same substance as original; added 'committed' gate — file must be in version control, not just local.
SPR-1	Security	No SecureNote plaintext shall appear in any JSON file, log file, core dump, or temporary file at any time. SecureNote JSON records shall contain only	Added log file and core dump to the list — original only mentioned 'file'.

ID	Type	Requirement Statement (Refined)	What Changed / Why
		ciphertext, the UUID, the title, and the created/modified timestamps.	Added explicit list of allowed fields in the serialized record.
SPR-2	Security	The EncryptionEngine shall be defined as an injectable interface (pure abstract class). No SecureNote, NoteManager, or FileStorage class shall hold a concrete crypto dependency in its public API. Unit tests shall be able to substitute a mock EncryptionEngine without linking a real crypto library.	Expanded to cover FileStorage explicitly — original mentioned NoteManager and SecureNote but not the persistence layer, which also handles encrypted data.
SPR-3	Security	Concurrent access to the same storage file by two application instances is undefined behavior in v1.0. The README shall document this limitation explicitly. Future versions may introduce file locking.	New requirement. Previously unaddressed. Concurrent access was an open question — now explicitly scoped out for v1.0.

Part 2 — Ambiguity Review

The following ambiguities were identified by submitting each requirement and acceptance criterion to a targeted audit asking: what is underspecified or behaviorally inconsistent? Only items where the original text left room for two different compliant implementations are listed.

Item	Ambiguity Found	Resolution Applied
FR-1 / US-01	Title length had no upper bound. Any string of non-whitespace characters was accepted. No implementation could determine when to reject.	Added 255-character maximum. Consistent with common filesystem and UI constraints. Becomes testable.
FR-1 / US-01	The original requirement said 'system-generated UUID' with no format constraint. RFC 4122 v4 is a common choice but was not specified — a sequential integer would technically comply.	Added RFC 4122 v4 explicitly. Locks format for serialization and interoperability.
FR-4b / US-04	Quarantine behavior was stated for a 'corrupt file' but not defined for a partially corrupt file (valid JSON structure, but one or more note records malformed). Whole-file quarantine on any bad record would discard all healthy notes.	Added per-record skip: valid records load, malformed records are individually skipped and logged.
FR-6 / US-06	Case-sensitivity of title search was explicitly deferred as 'an initial implementation choice to be documented.' This means the requirement itself had undefined	Resolved to case-insensitive. Rationale: user expectation for a note search tool. Removes ambiguity from the requirement text entirely.

Item	Ambiguity Found	Resolution Applied
	behavior — two compliant implementations could produce different results.	
FR-4a (new)	The storage file's absence at startup was not addressed. A missing file is a plausible first-run scenario but the original requirement only described the corrupt-file path.	Added FR-4a: absent file is normal; system starts empty and creates the file on first save.
FR-5 / US-05	Passphrase failure returned a single 'failure status.' A caller could not distinguish 'wrong passphrase' from 'empty input' from 'too short' — all had the same return.	Added FR-5a: typed error codes for passphrase failures. Enables callers (including tests) to assert the correct failure reason.

Part 3 — Edge-Case Review

Edge cases were identified by asking: what input or system state is neither explicitly allowed nor explicitly forbidden by the current requirement text? Each item is classified as covered (no change needed), resolved by an existing refined requirement, or added as a new constraint.

Requirement(s)	Edge Case	Disposition
FR-1, FR-2	Two notes with identical titles. No uniqueness constraint exists on title — only on UUID.	Explicitly allowed. Titles are not unique identifiers. Documented in FR-4 (UUID is the sole key). GTest case: create two notes with the same title, verify both are retrievable by distinct UUIDs.
FR-1	UUID collision from the generator. RFC 4122 v4 collision probability is negligible but not zero.	Treated as a programming error. NoteManager.add() shall assert that the UUID is not already present before insertion. On collision, log and return failure — do not silently overwrite.
FR-4, FR-4b	Storage file partially written due to process termination mid-save (e.g., power loss or SIGKILL).	FR-4b per-record quarantine handles malformed records. However, the save operation itself must be atomic: write to a temp file, then rename. Rename is added as an implementation note in FR-4. Not a new requirement — an implementation constraint derived from the existing quarantine requirement.
FR-3, FR-5	Deletion of a SecureNote while a decryption operation is in progress (hypothetical in a single-threaded build, but relevant for future threading).	Out of scope for v1.0 (single-threaded). SPR-3 documents that concurrent access is undefined. Logged as a known limitation.
FR-5, SPR-1	Passphrase contains null bytes or non-ASCII characters. std::string can hold null	FR-5 passphrase shall be treated as an opaque byte sequence of type std::string. Comparison and minimum-length check

Requirement(s)	Edge Case	Disposition
	bytes; a naive C-string comparison would truncate.	shall use .size(), not strlen(). Added as an implementation note under FR-5.
FR-6	Search performed on an empty collection (zero notes). No notes match any query.	FR-6 already requires empty-result to return an empty collection with no error. This case is covered. GTest case: search on empty NoteManager returns empty vector, no exception.
FR-2	Edit a TextNote's body only (title unchanged). The update should not reset or re-validate the existing title.	FR-2 requires atomic update semantics. Field-level updates are independent — editing body must not touch title. Acceptance criterion already covers this; no new requirement needed.
FR-4, NFR-1	Load from file produces a note whose UUID key already exists in the unordered_map from a prior in-session operation (e.g., load called twice).	FR-4 now requires that a duplicate UUID found during load is discarded and logged. Second load call should merge rather than duplicate.

Part 4 — How AI Helped Refine the Requirements

AI was used to audit the existing requirement set for gaps rather than to generate new content. Analysis was framed around what was underspecified or behaviorally inconsistent, which surfaced ambiguities and missing edge cases that were not obvious from reading the requirements in isolation. The resulting changes were tighter constraints, new sub-requirements, and explicit scope exclusions.